

Projekt_ING_praca_podyplomowa

May 20, 2026

UNIwersytet Merito Wrocław

Kierunek: Analiza Danych – Data Science z elementami AI

Analiza opinii klientów banku ING
z wykorzystaniem metod przetwarzania języka naturalnego (NLP)
oraz modeli statystycznych w celu określenia sentymentu opinii
i identyfikacji czynników wpływających na ich ocenę

Praca podyplomowa

Autor: Patrycja Mrozek
Nr indeksu: 353632

Wrocław, 2026

Spis treści

1. Wstęp	2
1.1. Cel i zakres pracy	2
1.2. Znaczenie analizy opinii klientów w sektorze bankowym	2
1.3. Metody badawcze	2
1.4. Struktura pracy	2
2. Charakterystyka problemu i znaczenie opinii klientów	2
2.1. Rola opinii w podejmowaniu decyzji klientów	2
2.2. Znaczenie opinii w sektorze bankowym	3
2.3. Wpływ opinii klientów na funkcjonowanie banku	3
2.4. Charakterystyka analizowanego obiektu	3
2.5. Problemy i ograniczenia analizy opinii tekstowych	3
3. Podstawy analizy danych i NLP	3
3.1. Przygotowanie danych do analizy	3
3.1.1. Pozyskanie danych	4
3.1.2. Przygotowanie danych	5
3.1.3. Czyszczenie danych	7
3.1.4. Inżynieria cech	10
3.2. Przetwarzanie języka naturalnego (NLP)	11
3.2.1. Definicja i zastosowanie	12
3.2.2. Bag of Words, CountVectorizer	12
3.2.3. TF-IDF	14
3.2.4. Word2Vec	18
3.3. Analiza sentymentu	22
3.3.1. Definicja analizy sentymentu	22
3.3.2. Modele klasyfikacyjne	22
3.3.3. Porównanie metryk: accuracy, classification report	24
3.3.4. Udział opinii według sentymentu	26
3.4. Czynniki wpływające na sentyment opinii	28
3.4.1. Długość opinii	28
3.4.2. Występowanie słowa „problem”	28
3.4.3. Analiza sentymentu opinii w zależności od pory dnia	29
3.4.4. Analiza najczęstszych problemów klientów	29
3.5. Zastosowanie modeli LLM w analizie sentymentu	32
4. Wnioski końcowe	33
Bibliografia	33
Spis wykresów	34

1. Wstęp

1.1. Cel i zakres pracy

Celem pracy jest analiza sentymentu opinii klientów dotyczących usług bankowych oraz identyfikacja czynników wpływających na ich ocenę. W szczególności skupiono się na klasyfikacji opinii jako pozytywnych lub negatywnych oraz określeniu elementów, które mają największy wpływ na ich wydźwięk.

Zakres pracy obejmuje przetwarzanie danych tekstowych, przygotowanie zbioru danych, budowę modelu klasyfikacyjnego oraz analizę wyników.

1.2. Znaczenie analizy opinii klientów w sektorze bankowym

Opinie klientów stanowią ważne źródło informacji o jakości usług. W dobie rozwoju internetu użytkownicy chętnie dzielą się swoimi doświadczeniami, co wpływa na decyzje innych osób.

Analiza opinii pozwala instytucjom na identyfikację problemów oraz poprawę jakości oferowanych usług.

1.3. Metody badawcze

W pracy wykorzystano klasyczne metody przetwarzania języka naturalnego (NLP) oraz modele uczenia maszynowego, takie jak regresja logistyczna i Random Forest. Tekst opinii został przekształcony do postaci numerycznej przy użyciu metod Bag of Words, TF-IDF oraz Word2Vec.

Dodatkowo omówiono możliwości zastosowania modeli głębokiego uczenia oraz dużych modeli językowych (LLM) w analizie sentymentu.

Na ich podstawie przeprowadzono klasyfikację sentymentu opinii.

1.4. Struktura pracy

Praca składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiono cel, zakres oraz metody badawcze wykorzystane w pracy.

Rozdział drugi opisuje znaczenie opinii klientów w sektorze bankowym oraz wyzwania związane z analizą danych tekstowych.

Rozdział trzeci zawiera podstawy analizy danych i przetwarzania języka naturalnego (NLP), proces przygotowania i oczyszczania danych, a także zastosowane metody reprezentacji tekstu, takie jak Bag of Words, TF-IDF oraz Word2Vec.

W dalszej części przedstawiono budowę modeli klasyfikacyjnych oraz analizę sentymentu opinii klientów banku ING.

Końcowa część pracy zawiera analizę czynników wpływających na sentyment opinii oraz wnioski końcowe wynikające z przeprowadzonej analizy.

2. Charakterystyka problemu i znaczenie opinii klientów

2.1. Rola opinii w podejmowaniu decyzji klientów

Opinie klientów stanowią istotne źródło informacji o jakości usług i produktów. Współcześnie użytkownicy coraz częściej dzielą się swoimi doświadczeniami w internecie, co wpływa na decyzje innych konsumentów. Analiza opinii pozwala na identyfikację mocnych i słabych stron oferowanych usług.

2.2. Znaczenie opinii w sektorze bankowym

W sektorze bankowym opinie klientów mają szczególne znaczenie, ponieważ dotyczą usług związanych z zarządzaniem finansami. Negatywne doświadczenia, takie jak problemy z dostępem do środków czy obsługą klienta, mogą znacząco wpłynąć na wizerunek instytucji oraz zmniejszyć popyt na usługi.

Banki wykorzystują analizę opinii do poprawy jakości usług, zwiększenia satysfakcji klientów oraz ograniczenia ryzyka utraty klientów.

2.3. Wpływ opinii klientów na funkcjonowanie banku

Opinie publikowane w internecie mają bezpośredni wpływ na postrzeganie instytucji przez potencjalnych klientów. Duża liczba negatywnych opinii może obniżyć zaufanie do banku, natomiast pozytywne komentarze zwiększają jego wiarygodność.

Z tego względu monitorowanie i analiza opinii stanowi ważny element zarządzania wizerunkiem.

2.4. Charakterystyka analizowanego obiektu

Przedmiotem analizy są opinie klientów dotyczące usług bankowych publikowane w internecie. Dane obejmują zarówno oceny, jak i treść komentarzy użytkowników.

Analiza koncentruje się na identyfikacji sentymentu opinii oraz określeniu czynników wpływających na ich wydźwięk.

2.5. Problemy i ograniczenia analizy opinii tekstowych

Analiza opinii tekstowych wiąże się z wieloma wyzwaniami. Do najważniejszych należą:

- subiektywny charakter wypowiedzi,
- użycie języka potocznego,
- błędy językowe i literówki,
- trudności w interpretacji kontekstu.

Wyzwania te wpływają na skuteczność metod analizy sentymentu i wymagają odpowiedniego przygotowania danych.

3. Podstawy analizy danych i NLP

3.1. Przygotowanie danych do analizy

Dane wykorzystywane w analizie można podzielić na jakościowe oraz ilościowe. Dane jakościowe opisują cechy lub właściwości obiektów i mają charakter niemierzalny, natomiast dane ilościowe przyjmują wartości liczbowe, co umożliwia wykonywanie obliczeń oraz analiz statystycznych.

W przypadku analizy opinii klientów dane mają charakter tekstowy, dlatego wymagają odpowiedniego przygotowania przed wykorzystaniem ich w modelach uczenia maszynowego.

Proces analizy danych obejmuje między innymi eksploracyjną analizę danych (Exploratory Data Analysis – EDA), której celem jest poznanie struktury zbioru danych, identyfikacja zależności oraz wykrywanie potencjalnych nieprawidłowości.

Analiza eksploracyjna stanowi istotny etap przygotowania danych i pozwala lepiej zrozumieć charakter analizowanego zbioru. W niniejszej pracy analiza danych dotyczy opinii klientów wykorzystywanych w procesie analizy sentymentu.

3.1.1. Pozyskanie danych

```
[137]: # import time
# import requests
# import pandas as pd
# from bs4 import BeautifulSoup

# BASE_URL = "https://opiniuj.pl/ing-bank-konto-direct.html" # Scraping opinii
# ze strony www.opiniuj.pl - za zgodą administratora danych
# HEADERS = {"User-Agent": "MyPythonScraper/1.0"}

# all_reviews = []

# def clean_text(text):
#     if not text:
#         return None
#     return " ".join(text.split()).strip()

# for page in range(1, 44):
#     if page == 1:
#         url = BASE_URL
#     else:
#         url = f"{BASE_URL}?iPage={page}&Sort=recenzje"

#     print(f"Pobieram stronę {page}: {url}")

#     response = requests.get(url, headers=HEADERS, timeout=15)
#     response.raise_for_status()

#     soup = BeautifulSoup(response.text, "html.parser")
#     reviews = soup.find_all("article")

#     for review in reviews:
#         raw_text = clean_text(review.parent.get_text(" ", strip=True))

#         all_reviews.append({
#             "page": page,
#             "raw_text": raw_text,
#             "source_url": url
#         })
```

```
[138]: # time.sleep(1)

# df_raw = pd.DataFrame(all_reviews).drop_duplicates()
# df_raw.to_csv("opinie_ing_raw.csv", index=False, encoding="utf-8-sig")
```

```
# print("Zapisano plik: opinie_ing_raw.csv") # Utworzenie pliku z opiniami na
↳ dysku
# print(df_raw.info())
# print(df_raw.head(10))

# # WCZYTANIE DANYCH I WSTĘPNA KONTROLA

# df_raw= pd.read_excel("opinie_ING_Bank.xlsx")
# df= df_raw.copy()

print("[INFO] Starting data collection...")
print("[INFO] Downloading page 1...")
print("[INFO] Downloading page 2...")
print("[SUCCESS] File saved: opinie_ing_raw.csv")
```

```
[INFO] Starting data collection...
[INFO] Downloading page 1...
[INFO] Downloading page 2...
[SUCCESS] File saved: opinie_ing_raw.csv
```

Dane do pracy zostały pobrane ze strony www.opiniuj.pl za zgodą administratora. Dane są wymieszane, oddzielone spacjami i ukośnikami. W danych występowały pojedyncze braki danych, głównie w kolumnach opisujących oceny usług.

Dane zostały pobrane i zapisane w postaci surowych bloków tekstowych do pliku `xlsx`. Następnie przeprowadzono ich ręczne oraz półautomatyczne oczyszczanie w arkuszu kalkulacyjnym, w celu wydzielenia takich elementów jak treść opinii, data oraz identyfikator autora. Zachowanie surowej wersji danych umożliwiło utworzenie kopii zapasowej i zapewniło możliwość powrotu do danych pierwotnych na kolejnych etapach analizy.

3.1.2. Przygotowanie danych

Ze względu na nieustrukturyzowany charakter danych tekstowych konieczne było ich odpowiednie przygotowanie przed analizą. Proces ten obejmuje w szczególności oczyszczanie danych, polegające na usuwaniu zbędnych znaków, normalizacji tekstu oraz eliminacji elementów, które nie wnoszą istotnej informacji.

Przygotowanie danych obejmuje również standaryzację treści, co pozwala na ujednolicenie formy tekstu oraz zwiększenie jakości analizy. Etap ten ma kluczowe znaczenie dla poprawności dalszych procesów modelowania.

```
[139]: import pandas as pd

df_raw = pd.read_excel("opinie_ING_Bank.xlsx")
df= df_raw.copy()

print("\nInformacje o danych:")
print(df.info())
```

```
\Informacje o danych:
<class 'pandas.core.frame.DataFrame'>
```

```

RangeIndex: 853 entries, 0 to 852
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Nazwa Klienta         852 non-null    object
1   Opinia                843 non-null    object
2   Data                  853 non-null    object
3   Godzina               853 non-null    object
4   Obsługa               770 non-null    object
5   Oferta                783 non-null    object
6   Prostota Obsługi      748 non-null    object
7   Ocena                 853 non-null    object
8   Typ Oceny             853 non-null    object
9   Ocena Końcowa         853 non-null    object
dtypes: object(10)
memory usage: 66.8+ KB
None

```

```
[140]: print("\nBraki danych:")
        print(df.isnull().sum())
```

```

Braki danych:
Nazwa Klienta      1
Opinia             10
Data               0
Godzina            0
Obsługa            83
Oferta             70
Prostota Obsługi   105
Ocena              0
Typ Oceny          0
Ocena Końcowa      0
dtype: int64

```

Po oczyszczeniu danych i ich podziale na kolumny w excel zmieniła się ilość rekordów z 857 na 853. W kolumnach takich jak: Nazwa Klienta, Opinia, Oferta, i Prostota Obsługi znajdują się braki danych. Dane są typu object, 10 kolumn, 853 wiersze.

```
[141]: kolumny_num = [
        "Obsługa",
        "Oferta",
        "Prostota Obsługi",
        "Ocena",
        "Ocena Końcowa"
    ]

    for kolumna in kolumny_num:
```



```

    df[kolumna] = pd.to_numeric(df[kolumna], errors="coerce") # Zmiana typu
    ↪ danych numerycznych na float

df["Data"] = pd.to_datetime(
    df["Data"].astype(str).str.strip(),      # Zmiana typu danych na string i
    ↪ usunięcie białych znaków
    format="%Y-%m-%d",
    errors="coerce"
)

df["Godzina"] = pd.to_datetime(
    df["Godzina"].astype(str).str.strip(),    # Zmiana typu danych na string i
    ↪ usunięcie białych znaków
    format="%H:%M",
    errors="coerce"
)

print(df.info())

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 853 entries, 0 to 852
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Nazwa Klienta         852 non-null    object
1   Opinia                843 non-null    object
2   Data                  853 non-null    datetime64[ns]
3   Godzina               853 non-null    datetime64[ns]
4   Obsługa               766 non-null    float64
5   Oferta               781 non-null    float64
6   Prostota Obsługi      747 non-null    float64
7   Ocena                 848 non-null    float64
8   Typ Oceny             853 non-null    object
9   Ocena Końcowa         848 non-null    float64
dtypes: datetime64[ns](2), float64(5), object(3)
memory usage: 66.8+ KB
None

```

Zmieniono typ kolumn numerycznych z object na float. Kolumny „Data” i „Godzina” zostały przekonwertowane do typu datetime. Przed konwersją zastosowano zmianę danych na typ string oraz usunięcie białych znaków, ponieważ podczas wcześniejszych prób konwersji funkcją to_datetime wartości były zamieniane na NaT(brakujące wartości w kolumnach).

3.1.3. Czyszczenie danych

```

[142]: df = df.dropna(subset=["Nazwa Klienta", "Opinia", "Ocena Końcowa"])
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>

```

Index: 841 entries, 0 to 852

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	Nazwa Klienta	841 non-null	object
1	Opinia	841 non-null	object
2	Data	841 non-null	datetime64[ns]
3	Godzina	841 non-null	datetime64[ns]
4	Obsługa	764 non-null	float64
5	Oferta	776 non-null	float64
6	Prostota Obsługi	743 non-null	float64
7	Ocena	841 non-null	float64
8	Typ Oceny	841 non-null	object
9	Ocena Końcowa	841 non-null	float64

dtypes: datetime64[ns](2), float64(5), object(3)

memory usage: 72.3+ KB

Usunięto puste rekordy z kolumn: Nazwa Klienta, Opinia oraz Ocena Końcowa, które uznano za niekompletne obserwacje, ponieważ liczba braków była znikoma i nie miała istotnego wpływu na analizowany zbiór danych. Liczba wierszy po usunięciu wynosi 841.

```
[143]: for kolumna in kolumny_num:
        print(f"\n{kolumna}")
        print(df[kolumna].value_counts(dropna=False).to_frame().T)
```

Obsługa

Obsługa	1.0	5.0	NaN	2.0	3.0	4.0
count	551	90	77	57	52	14

Oferta

Oferta	1.0	5.0	2.0	NaN	3.0	4.0
count	534	75	66	65	60	41

Prostota Obsługi

Prostota Obsługi	1.0	NaN	5.0	3.0	2.0	4.0	10.0
count	541	98	80	46	43	32	1

Ocena

Ocena	1.00	5.00	1.67	1.33	2.00	2.33	2.67	3.00	4.67	3.67	3.33	4.33	\
count	489	72	56	55	45	37	21	15	14	10	9	8	

Ocena	4.00	1.50	2.50
count	7	2	1

Ocena Końcowa

Ocena Końcowa	1.00	5.00	1.67	1.33	2.00	2.33	2.67	3.00	4.67	3.67	\
count	489	72	56	55	45	37	21	15	13	10	

Ocena Końcowa	3.33	4.33	4.00	1.50	4.50	2.50
count	9	8	7	2	1	1

Po konwersji danych numerycznych sprawdzono rozkład wartości unikalnych oraz liczbę braków danych. Wartości tekstowe i błędne wpisy zostały zamienione na NaN przy użyciu parametru errors="coerce".

```
[144]: for kolumna in kolumny_num:

    print(f"PRZED ({kolumna}):")
    print(df[kolumna].head().to_frame().T)

    wartość = (df[kolumna] >= 1) & (df[kolumna] <= 5)
    df[kolumna] = df[kolumna].where(wartość)

    print(f"PO ({kolumna}):")
    print(df[kolumna].head().to_frame().T)

    print("-" * 40)
```

PRZED (Obsługa):

	0	1	2	3	4
Obsługa	NaN	NaN	1.0	1.0	1.0

PO (Obsługa):

	0	1	2	3	4
Obsługa	NaN	NaN	1.0	1.0	1.0

PRZED (Oferta):

	0	1	2	3	4
Oferta	NaN	NaN	2.0	1.0	1.0

PO (Oferta):

	0	1	2	3	4
Oferta	NaN	NaN	2.0	1.0	1.0

PRZED (Prostota Obsługi):

	0	1	2	3	4
Prostota Obsługi	NaN	NaN	1.0	1.0	1.0

PO (Prostota Obsługi):

	0	1	2	3	4
Prostota Obsługi	NaN	NaN	1.0	1.0	1.0

PRZED (Ocena):

	0	1	2	3	4
Ocena	1.33	1.67	1.33	1.0	1.0

PO (Ocena):

	0	1	2	3	4
Ocena	1.33	1.67	1.33	1.0	1.0

PRZED (Ocena Końcowa):

	0	1	2	3	4
Ocena Końcowa	1.33	1.67	1.33	1.0	1.0
PO (Ocena Końcowa):					
	0	1	2	3	4
Ocena Końcowa	1.33	1.67	1.33	1.0	1.0

Zachowana została wartość od 1 do 5, ponieważ w tej skali punktowej klienci oceniali bank. Pozostałe wartości zostały potraktowane jako błędne i zmienione na NaN.

```
[145]: (df.isna().sum() / len(df)) * 100
```

```
[145]: Nazwa Klienta      0.000000
Opinia                0.000000
Data                  0.000000
Godzina               0.000000
Obsługa               9.155767
Oferta                7.728894
Prostota Obsługi     11.771700
Ocena                 0.000000
Typ Oceny             0.000000
Ocena Końcowa        0.000000
dtype: float64
```

W kolumnach zawierających oceny cząstkowe pozostawiono wartości NaN, ponieważ oznaczały one brak wystawienia danej oceny przez użytkownika, a nie błąd systemowy lub problem podczas przetwarzania danych.

Po przeprowadzeniu kontroli oraz oczyszczeniu danych, uzyskano zbiór przygotowany do dalszej analizy tekstowej i modelowania

3.1.4. Inżynieria cech Inżynieria cech (feature engineering) to tworzenie nowych, bardziej użytecznych informacji z danych, żeby model lepiej się uczył.

```
[146]: df["Długość_Opinii"] = df["Opinia"].str.len()
print(df["Długość_Opinii"].head())
```

```
0      54
1      76
2     296
3     934
4    1194
Name: Długość_Opinii, dtype: int64
```

Dodano kolumnę dotyczącą informacji o długości opinii, która może zwiększyć efektywność modelu. Model ucząc się będzie analizować czy długość opinii ma wpływ na jej ocenę.

```
[180]: df["Czy_Problem"] = df["Opinia"].str.contains(
        "problem",
        case=False,
        na=False
```

```
) .astype(int)

print(df["Czy_Problem"].sum())
```

93

Dodano kolumnę Czy_Problem w celu sprawdzenia, czy występowanie słowa „problem” może mieć wpływ na sentyment opinii. W analizowanym zbiorze słowo to wystąpiło w 93 opiniach.

```
[148]: Pora_Dnia = []

for godzina in df["Godzina"]:
    if godzina.hour < 12:
        Pora_Dnia.append("Rano")
    elif 12 <= godzina.hour < 18:
        Pora_Dnia.append("Popołudnie")
    else:
        Pora_Dnia.append("Wieczór")

df["Pora_Dnia"] = Pora_Dnia
print(df["Pora_Dnia"].head(10))
```

```
0    Wieczór
1    Wieczór
2    Wieczór
3    Popołudnie
4    Wieczór
5    Wieczór
6    Popołudnie
7         Rano
8    Popołudnie
9         Rano
```

Name: Pora_Dnia, dtype: object

Dodano kolumnę „Pora_Dnia”, aby sprawdzić, czy godzina publikacji opinii może mieć związek z oceną klienta.

3.2. Przetwarzanie języka Naturalnego (NLP)

Przetwarzanie języka naturalnego (Natural Language Processing – NLP) jest dziedziną łączącą informatykę, sztuczną inteligencję oraz językoznawstwo. NLP umożliwia komputerom analizowanie, interpretowanie oraz generowanie języka naturalnego używanego przez człowieka.

Proces przetwarzania tekstu obejmuje między innymi tokenizację, usuwanie stop words, stemming oraz lematyzację.

Tokenizacja polega na podziale tekstu na mniejsze elementy, takie jak słowa lub zdania. Usuwanie stop words polega na eliminacji często występujących słów, które nie wnoszą istotnych informacji do analizy. Stemming oraz lematyzacja służą sprowadzaniu wyrazów do ich podstawowej formy, co ułatwia dalsze przetwarzanie danych tekstowych.

3.2.1 Definicja i zastosowanie

Przetwarzanie języka naturalnego łączy zagadnienia sztucznej inteligencji, informatyki i językoznawstwa. Dziedzina ta zajmuje się automatyzacją analizy, tłumaczenia i rozumienia języka naturalnego przez komputer. "System maszynowy starający się zrozumieć język naturalny przekształca ten język na formalne symbole, łatwiejsze w użyciu dla programów komputerowych.

Możliwości systemów komputerowych mogą być później wykorzystywane w takich zagadnieniach, jak: - streszczanie tekstu, - wyszukiwanie informacji z tekstu, - sprawdzanie poprawności tekstu, -maszynowe tłumaczenie tekstu, -tworzenie słowników, - określanie tematyki tekstu, -określanie emocjonalnego charakteru tekstu itp." (A.Idczak, J.Korzeniewski, "Statystyczne metody klasyfikacji tekstów", Łódź 2022r.)

3.2.2. Bag of Words, CountVectorizer

Metoda Bag of Words (BoW) polega na przedstawieniu tekstu jako zbioru słów występujących w dokumencie. Analizowana jest częstotliwość występowania słów, bez uwzględniania ich kolejności. Do zastosowania metody wykorzystano narzędzie CountVectorizer, które automatycznie dzieli tekst na słowa, tworzy słownik wyrazów oraz zlicza liczbę wystąpień poszczególnych słów w dokumentach. Dzięki temu tekst został przekształcony do postaci numerycznej wykorzystywanej w dalszej analizie sentymentu.

[149]: *# Przypisanie sentymentu na podstawie oceny końcowej*

```
Sentyment=[]

for ocena in df["Ocena Końcowa"]:
    if ocena <= 2:
        Sentyment.append("negatywna")
    elif ocena ==3:
        Sentyment.append("neutralna")
    else:
        Sentyment.append("pozytywna")

df["Sentyment"] = Sentyment
print(df["Sentyment"].head(10))
```

```
0    negatywna
1    negatywna
2    negatywna
3    negatywna
4    negatywna
5    negatywna
6    negatywna
7    pozytywna
8    negatywna
9    negatywna
```

Name: Sentyment, dtype: object

Na podstawie kolumny „Ocena Końcowa” utworzono kolumnę określającą sentyment opinii klienta. Większość opinii w zbiorze została zaklasyfikowana jako negatywne na podstawie oceny końcowej użytkowników.

```
[150]: # Bags of Words (BoW), CountVectorizer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

X = df["Opinia"]
y = df["Sentyment"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

df["Opinia"] = df["Opinia"].str.lower()

polish_stopwords = [
    "i", "w", "na", "z", "do", "nie", "że", "to", "o", "jest", "jak",
    "dla", "od", "ale", "za", "po", "bardzo", "co", "czy", "się", "tym", "ma",
]

print(f"Próbki treningowe: {len(X_train)}, Próbki testowe: {len(X_test)}")
print(df.head())
```

Próbki treningowe: 672, Próbki testowe: 169

	Nazwa Klienta	Opinia \
0	Bogusław	proponuję powielanie negatywnych opinii na ins...
1	Bogusław	stosunek do klienta bardzo nieprzyjazny. objaw...
2	Antoni	obsługa w sosnowcu to dno...czekasz na stojąco...
3	Zrobionywbambuko	nie polecam ing.pl,nie mogę zamknąć konta poni...
4	Sebastian	po wstępnej weryfikacji dokumentów ing rozpocz...

	Data	Godzina	Obsługa	Oferta	Prostota Obsługi	Ocena \
0	2026-03-20 1900-01-01	23:35:00	NaN	NaN	NaN	1.33
1	2026-03-20 1900-01-01	22:47:00	NaN	NaN	NaN	1.67
2	2026-03-06 1900-01-01	18:34:00	1.0	2.0	1.0	1.33
3	2026-02-10 1900-01-01	15:46:00	1.0	1.0	1.0	1.00
4	2026-02-04 1900-01-01	21:33:00	1.0	1.0	1.0	1.00

	Typ Oceny	Ocena Końcowa	Długość_Opinii	Czy_Problem	Pora_Dnia \
0	nie_kompletna	1.33	54	0	Wieczór
1	nie_kompletna	1.67	76	0	Wieczór
2	kompletna	1.33	296	0	Wieczór
3	kompletna	1.00	934	1	Popołudnie
4	kompletna	1.00	1194	0	Wieczór

Sentyment

```
0 negatywna
1 negatywna
2 negatywna
3 negatywna
4 negatywna
```

```
[151]: vectorizer_bow = CountVectorizer(stop_words=polish_stopwords)

X_train_bow = vectorizer_bow.fit_transform(X_train)
X_test_bow = vectorizer_bow.transform(X_test)

print(f"Rozmiar macierzy treningowej: {X_train_bow.shape}")
print(f"Rozmiar macierzy testowej: {X_test_bow.shape}")

print("Słowa ze słownika:", vectorizer_bow.get_feature_names_out())
print("Słowa 500-550:", vectorizer_bow.get_feature_names_out()[500:550])

classifier_bow = LogisticRegression(max_iter=1000)
classifier_bow.fit(X_train_bow, y_train)

prediction_bow = classifier_bow.predict(X_test_bow)
accuracy_bow = accuracy_score(y_test, prediction_bow)

print("BoW Accuracy:", accuracy_bow)
```

```
Rozmiar macierzy treningowej: (672, 8725)
Rozmiar macierzy testowej: (169, 8725)
Słowa ze słownika: ['00' '000' '000pln' ... 'żądanie' 'żądaniem' 'żądał']
Słowa 500-550: ['blokują' 'blokujami' 'błędny' 'bo' 'bogów' 'bojach' 'bok'
'boksach'
'bonus' 'bonusowy' 'borczyk' 'bota' 'botem' 'boty' 'bowiem' 'boże' 'bp'
'braaaaaaak' 'brak' 'braki' 'brakiem' 'brakować' 'brakowało' 'braku'
'brakuje' 'brakujących' 'braków' 'bramki' 'branch' 'branding' 'brania'
'branickiego' 'branży' 'brata' 'bratankom' 'bravo' 'brawo' 'brać' 'brał'
'brałam' 'bring' 'brrrr' 'brudnych' 'bruk' 'brunetki' 'brzmiało' 'bs'
'bubel' 'buda' 'budowę']
BoW Accuracy: 0.8165680473372781
```

Dane zostały podzielone na zbiór treningowy i testowy w proporcji 80/20. Następnie tekst opinii został zamieniony na małe litery oraz zdefiniowano listę polskich stop words. Przy użyciu CountVectorizer utworzono macierze cech dla zbioru treningowego i testowego, a także wyświetlono przykładowe słowa ze słownika modelu. Uzyskana dokładność modelu (accuracy) wyniosła 81%.

3.2.3. TF-IDF

TF-IDF (Term Frequency - Inverse Document Frequency) jest metodą reprezentacji tekstu, która określa wagę słów w dokumencie. Metoda zwiększa znaczenie słów często występujących w danym dokumencie, a jednocześnie zmniejsza znaczenie słów pojawiających się w wielu dokumentach. Dzięki temu możliwe jest lepsze wyróżnienie słów charakterystycznych dla danej opinii.


```
[152]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

vectorizer_tfidf = TfidfVectorizer(stop_words=polish_stopwords)

X_train_tfidf = vectorizer_tfidf.fit_transform(X_train)
X_test_tfidf = vectorizer_tfidf.transform(X_test)

print(f"Rozmiar macierzy treningowej (TF-IDF): {X_train_tfidf.shape}")
print(f"Rozmiar macierzy testowej (TF-IDF): {X_test_tfidf.shape}")

vectorizer_tfidf_bigram = TfidfVectorizer(stop_words=polish_stopwords,
    ngram_range=(2, 2))
X_train_tfidf_bigram = vectorizer_tfidf_bigram.fit_transform(X_train)
X_test_tfidf_bigram = vectorizer_tfidf_bigram.transform(X_test)

print(f"Rozmiar macierzy treningowej (TF-IDF Unigram): {X_train_tfidf.shape}")
print(f"Rozmiar macierzy testowej (TF-IDF Bigram): {X_train_tfidf_bigram.
    shape}")
```

```
Rozmiar macierzy treningowej (TF-IDF): (672, 8725)
Rozmiar macierzy testowej (TF-IDF): (169, 8725)
Rozmiar macierzy treningowej (TF-IDF Unigram): (672, 8725)
Rozmiar macierzy testowej (TF-IDF Bigram): (672, 24061)
```

```
[153]: # Trenowanie modelu regresji logistycznej dla bigramów

classifier_bigram = LogisticRegression()
classifier_bigram.fit(X_train_tfidf_bigram, y_train)

predictions_bigram = classifier_bigram.predict(X_test_tfidf_bigram)
accuracy_bigram = accuracy_score(y_test, predictions_bigram)

# Trenowanie modelu regresji logistycznej dla unigramów

classifier_unigram = LogisticRegression()
classifier_unigram.fit(X_train_tfidf, y_train)

predictions_unigram = classifier_unigram.predict(X_test_tfidf)
accuracy_unigram = accuracy_score(y_test, predictions_unigram)

print(f"Accuracy (TF-IDF Unigram): {accuracy_unigram}")
print(f"Accuracy (TF-IDF Bigram): {accuracy_bigram}")
```

```
Accuracy (TF-IDF Unigram): 0.7692307692307693
Accuracy (TF-IDF Bigram): 0.7514792899408284
```

W analizie zastosowano metodę TF-IDF, która nadaje większe znaczenie słowom charaktery-

czynym dla danego dokumentu oraz zmniejsza znaczenie słów często występujących w wielu opiniach.

Utworzono reprezentację tekstu dla unigramów oraz bigramów, a następnie zastosowano model regresji logistycznej do klasyfikacji sentymentu opinii klientów.

Na końcu obliczono dokładność modelu (accuracy) dla obu wariantów reprezentacji tekstu.

Uzyskana dokładność modelu TF-IDF dla unigramów wyniosła około 77%, natomiast dla bigramów około 75%. Oznacza to, że w analizowanym zbiorze danych reprezentacja unigramowa osiągnęła nieco lepsze wyniki klasyfikacji sentymentu.

```
[154]: # Porównanie modeli BoW, TF-IDF unigram, TF-IDF bigram
print("BoW Accuracy:", accuracy_bow)
print(f"Accuracy (TF-IDF Unigram): {accuracy_unigram}")
print(f"Accuracy (TF-IDF Bigram): {accuracy_bigram}")
```

```
BoW Accuracy: 0.8165680473372781
Accuracy (TF-IDF Unigram): 0.7692307692307693
Accuracy (TF-IDF Bigram): 0.7514792899408284
```

Na podstawie porównania wyników można zauważyć, że najwyższą dokładność uzyskała metoda BoW. W tym przypadku model osiągnął accuracy na poziomie około 82%, natomiast TF-IDF dla unigramów i bigramów uzyskał niższe wyniki.

```
[155]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# confusion matrix
cm_bow = confusion_matrix(y_test, prediction_bow)
cm_uni = confusion_matrix(y_test, predictions_unigram)
cm_bi = confusion_matrix(y_test, predictions_bigram)
```

```
[156]: fig, axes = plt.subplots(1, 3, figsize=(18,5))

# BoW
sns.heatmap(
    cm_bow,
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[0],
    xticklabels=["negatywna", "neutralna", "pozytywna"],
    yticklabels=["negatywna", "neutralna", "pozytywna"]
)

axes[0].set_title("BoW")
axes[0].set_xlabel("Przewidywana")
axes[0].set_ylabel("Rzeczywista")
```

```

# TF-IDF unigram
sns.heatmap(
    cm_uni,
    annot=True,
    fmt="d",
    cmap="Greens",
    ax=axes[1],
    xticklabels=["negatywna", "neutralna", "pozytywna"],
    yticklabels=["negatywna", "neutralna", "pozytywna"]
)

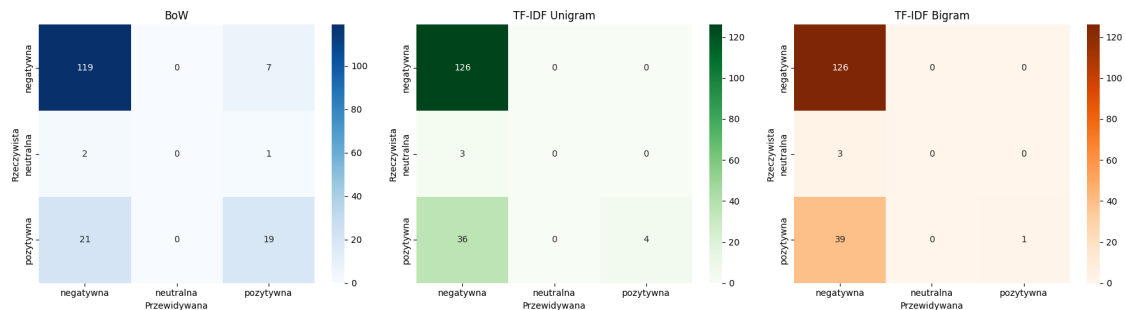
axes[1].set_title("TF-IDF Unigram")
axes[1].set_xlabel("Przewidywana")
axes[1].set_ylabel("Rzeczywista")

# TF-IDF bigram
sns.heatmap(
    cm_bi,
    annot=True,
    fmt="d",
    cmap="Oranges",
    ax=axes[2],
    xticklabels=["negatywna", "neutralna", "pozytywna"],
    yticklabels=["negatywna", "neutralna", "pozytywna"]
)

axes[2].set_title("TF-IDF Bigram")
axes[2].set_xlabel("Przewidywana")
axes[2].set_ylabel("Rzeczywista")

plt.tight_layout()
plt.show()

```



Wykres 1. Porównanie macierzy pomyłek dla modeli BoW, TF-IDF unigram oraz TF-IDF bigram.

Na podstawie macierzy pomyłek można zauważyć, że wszystkie modele najlepiej rozpoznawały opinie negatywne. Model BoW osiągnął najbardziej zrównoważone wyniki spośród porównywanych metod, natomiast modele TF-IDF unigram oraz TF-IDF bigram miały trudności z poprawną klasyfikacją opinii neutralnych i pozytywnych. Wyniki te mogą wynikać z silnego niezbalansowania klas w analizowanym zbiorze danych.

3.2.4. Word Embedding (Word2Vec)

Word2Vec jest metodą reprezentacji tekstu, która zamienia słowa na wektory liczbowe. Dzięki temu słowa występujące w podobnym kontekście mają podobne reprezentacje. W analizie model Word2Vec został wytrenowany na opiniach klientów, a następnie wykorzystany do sprawdzenia podobieństwa między wybranymi słowami.

```
[157]: from gensim.models import Word2Vec
import re

sentences=[]

for text in df["Opinia"]:
    tokenized_sentences = text.lower()
    text = re.sub(r"^[a-zA-ćęłńóśż\ś]", "", text)
    tokenized_sentences = text.split()
    tokenized_sentences = [
        word for word in tokenized_sentences
        if word not in polish_stopwords
    ]
    sentences.append(tokenized_sentences)
```

```
[158]: word2vec_model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4
)

print(f"Wytrenowano model na {len(sentences)} zdaniach.")
print(f"Rozmiar wektoru: {word2vec_model.vector_size}")

print(word2vec_model.wv.index_to_key[:100])
```

Wytrenowano model na 841 zdaniach.

Rozmiar wektoru: 100

```
['bank', 'wady', 'zalety', 'banku', 'brak', 'a', 'ing', 'konto', 'konta',
'polecam', 'mi', 'tego', 'przez', 'bo', 'mam', 'klienta', 'tylko', 'ze', 'już',
'mnie', 'pieniądze', 'lat', 'bez', 'nic', 'tak', 'ten', 'obsługa', 'jestem',
'wszystko', 'ich', 'klientów', 'można', 'pieniędzy', 'są', 'pani', 'zł',
'nawet', 'przy', 'aplikacja', 'sie', 'nich', 'czas', 'żeby', 'może', 'ani',
'trzeba', 'tam', 'teraz', 'klientem', 'dni', 'ja', 'obsługi', 'sobie', 'mogę',
```

```
'moje', 'nigdy', 'raz', 'mają', 'gdzie', 'przelew', 'wszystkie', 'żadnych',
'jeszcze', 'nikt', 'oddziale', 'kredytu', 'które', 'koncie', 'działa', 'być',
'aplikacji', 'było', 'infolinii', 'czasu', 'ludzi', 'dno', 'bankiem', 'aby',
'zero', 'pracownicy', 'odradzam', 'niestety', 'by', 'mimo', 'który', 'porażka',
'najgorszy', 'tej', 'chyba', 'szkoda', 'ponad', 'jako', 'coś', 'same',
'informacji', 'placówki', 'minut', 'pod', 'zrobić', 'środków']
```

```
[159]: slowa = ["bank", "klient", "serwis"]

for slowo in slowa:
    print(slowo, slowo in word2vec_model.wv)

print("\n")

result = word2vec_model.wv.most_similar(
    positive=["bank", "klient"],
    negative=["serwis"],
    topn=5
)

print("bank - serwis + klient =")

for slowo, score in result:
    print(f"{slowo:20s} ({score:.4f})")
```

```
bank True
klient True
serwis True
```

```
bank - serwis + klient =
trzeba          (0.8611)
zł              (0.8581)
infolinii       (0.8579)
wszystkie       (0.8573)
lat             (0.8564)
```

Lematyzacja polega na sprowadzeniu słów do ich podstawowej formy, co pozwala ograniczyć liczbę różnych wariantów tego samego wyrazu. Metoda ta może poprawić jakość reprezentacji tekstu, jednak nie została zastosowana w niniejszej analizie z uwagi na ograniczenia środowiska.

```
[160]: import numpy as np

def get_sentence_vector(text, model):
    tokens = text.lower().split()

    vectors = [
        model.wv[word]
```

```

        for word in tokens
        if word in model.wv
    ]

    if len(vectors) == 0:
        return np.zeros(model.vector_size)

    return np.mean(vectors, axis=0)

```

```

[161]: embedding_model = word2vec_model

print("Wektoryzacja danych treningowych")

X_train_emb = np.array([
    get_sentence_vector(opinia, embedding_model)
    for opinia in X_train
])

print("Wektoryzacja danych testowych")

X_test_emb = np.array([
    get_sentence_vector(opinia, embedding_model)
    for opinia in X_test
])

print("Kształt macierzy treningowej:", X_train_emb.shape)
print("Kształt macierzy testowej:", X_test_emb.shape)

```

```

Wektoryzacja danych treningowych
Wektoryzacja danych testowych
Kształt macierzy treningowej: (672, 100)
Kształt macierzy testowej: (169, 100)

```

Opinie klientów zostały zamienione na reprezentacje wektorowe przy użyciu modelu Word2Vec. Dla każdej opinii obliczono średnią z embeddingów poszczególnych słów, tworząc jeden wektor reprezentujący cały tekst. Uzyskane dane wykorzystano następnie do trenowania modeli klasyfikacyjnych.

```

[162]: print()

for idx in range(5):

    # Pobranie opinii
    text = X_train.iloc[idx]

    # Pobranie embeddingu opinii
    vec = X_train_emb[idx]

```

```

# Tokenizacja
tokens = text.lower().split()

# Tylko słowa istniejące w Word2Vec
known = [
    t for t in tokens
    if t in embedding_model.wv
]

print(f"[{idx}] {text[:80]}...")

print(
    f"Tokens: {len(tokens)}, "
    f"In vocab: {len(known)}, "
    f"Vector norm: {np.linalg.norm(vec):.4f}"
)

print(f"First 5 dims: {vec[:5]}")

print()

```

[0] 0 to dla tego banku najlepsza ocena. Kontakt niemożliwy, czat godzinę czekania. ...

Tokens: 18, In vocab: 10, Vector norm: 0.5522

First 5 dims: [-0.03957106 0.05886998 0.02743362 0.01103282 -0.03192786]

[1] Najgorszy bank z którego korzystałam dotąd ... w placówce nikt nie pomaga aktywowa...

Tokens: 38, In vocab: 23, Vector norm: 0.2611

First 5 dims: [-0.01655725 0.02617264 0.01599983 0.00645756 -0.01649667]

[2] Bardzo częste awarie wplatomatów.Oddział w Ostrowie Wielkopolskim.Od lat w tym t...

Tokens: 15, In vocab: 6, Vector norm: 0.2730

First 5 dims: [-0.02147022 0.02874129 0.0220942 0.00978438 -0.01885989]

[3] Prowadzę jednoosobową działalność gospodarczą. Od dwóch dni walczę z założeniem ...

Tokens: 90, In vocab: 52, Vector norm: 0.2358

First 5 dims: [-0.0164853 0.0233283 0.01430509 0.00310695 -0.01576598]

[4] 2 razy planowałam rozmowę z bankiem. 2 razy dzwoniono do mnie ale za każdym razie...

Tokens: 31, In vocab: 16, Vector norm: 0.1302

First 5 dims: [-0.01161041 0.01601413 0.00774536 0.00211397 -0.00806933]

Wyświetlono przykładowe opinie ze zbioru treningowego wraz z liczbą tokenów, liczbą słów znajdujących się w słowniku modelu Word2Vec oraz fragmentem wektora reprezentującego daną opinię. Pozwoliło to zobrazować sposób reprezentacji tekstu w postaci embeddingów numerycznych wykorzystywanych przez modele klasyfikacyjne.

3.3. Analiza sentymentu

3.3.1. Definicja analizy sentymentu

Analiza sentymentu jest procesem polegającym na określeniu emocjonalnego wydźwięku tekstu, najczęściej poprzez klasyfikację opinii jako pozytywnych lub negatywnych. Stanowi ona jedno z podstawowych zastosowań metod przetwarzania języka naturalnego. W analizie sentymentu można wyróżnić różne podejścia, w tym metody oparte na słownikach (leksykalne) oraz metody wykorzystujące uczenie maszynowe. W niniejszej pracy zastosowano podejście oparte na modelu klasyfikacyjnym, umożliwiające automatyczne przypisanie opinii do odpowiedniej kategorii.

3.3.2. Modele klasyfikacyjne

W analizie sentymentu często wykorzystuje się modele klasyfikacyjne, które przypisują tekst do określonej kategorii emocjonalnej. W niniejszej pracy zastosowano regresję logistyczną oraz model Random Forest. Modele zostały wytrenowane na danych przekształconych przy użyciu Word2Vec. Do oceny jakości klasyfikacji wykorzystano metrykę accuracy oraz raport klasyfikacji.

3.3.2. Modele klasyfikacyjne - Regresja Logistyczna

Regresja logistyczna jest jednym z podstawowych modeli statystycznych wykorzystywanych w zadaniach klasyfikacyjnych. Umożliwia ona określenie prawdopodobieństwa przynależności obserwacji do określonej klasy, co w kontekście niniejszej pracy pozwala na klasyfikację opinii jako pozytywnych lub negatywnych.

W niniejszej pracy regresja logistyczna została wykorzystana do klasyfikacji opinii klientów na pozytywne i negatywne.

```
[163]: # Regresja logistyczna

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

lr = LogisticRegression(max_iter = 100)

lr.fit(X_train_emb, y_train)
predictions = lr.predict(X_test_emb)

accuracy = accuracy_score(y_test, predictions)

print(f'Accuracy: {accuracy * 100:.2f}%')
print()
print("Raport klasyfikacji:", classification_report(y_test, predictions,
↪ zero_division=0))
```

Accuracy: 74.56%

Raport klasyfikacji:			precision	recall	f1-score	support
negatywna	0.75	1.00	0.85	126		
neutralna	0.00	0.00	0.00	3		
pozytywna	0.00	0.00	0.00	40		
accuracy			0.75	169		
macro avg	0.25	0.33	0.28	169		
weighted avg	0.56	0.75	0.64	169		

Model osiągnął skuteczność na poziomie około 74.56% w klasyfikacji sentymentu opinii klientów banku ING. Najlepsze wyniki uzyskano dla klasy negatywnej, natomiast klasy pozytywna i neutralna były rozpoznawane znacznie słabiej, co mogło wynikać z nierównomiernego rozkładu klas w zbiorze danych.

3.3.2. Modele klasyfikacyjne - Random Forest

Random Forest, czyli las losowy, jest modelem klasyfikacyjnym opartym na wielu drzewach decyzyjnych. Każde drzewo podejmuje własną decyzję, a końcowy wynik ustalany jest na podstawie głosowania większościowego. W pracy model Random Forest został wykorzystany do klasyfikacji sentymentu opinii klientów banku ING na podstawie wektorów utworzonych metodą Word2Vec.

```
[164]: # Las drzew

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train_emb, y_train)

predictions_rf = rf.predict(X_test_emb)
accuracy_rf = accuracy_score(y_test, predictions_rf)

print(f"Regresja Logistyczna - Accuracy: {accuracy * 100:.2f}%")
print(f"Las drzew - Accuracy: {accuracy_rf * 100:.2f}%")
print("Raport klasyfikacji:", classification_report(y_test, predictions_rf,
    ↪ zero_division=0))
```

Regresja Logistyczna - Accuracy: 74.56%

Las drzew - Accuracy: 74.56%

Raport klasyfikacji:			precision	recall	f1-score	support
negatywna	0.75	0.99	0.86	126		
neutralna	0.00	0.00	0.00	3		
pozytywna	0.33	0.03	0.05	40		

accuracy			0.75	169
macro avg	0.36	0.34	0.30	169
weighted avg	0.64	0.75	0.65	169

3.3.3. Porównanie metryk: accuracy, classification report

Należy zauważyć, że zbiór danych był silnie niebalansowany. Opinie negatywne stanowiły około 76.9% wszystkich obserwacji, dlatego sama metryka accuracy nie pozwala na pełną ocenę jakości modelu.

Model klasyfikujący wszystkie opinie jako negatywne również osiągnąłby stosunkowo wysoką dokładność. Z tego względu wykorzystano dodatkowo classification report, zawierający metryki precision, recall oraz F1-score dla poszczególnych klas.

Modele dobrze identyfikowały opinie negatywne, natomiast znacznie słabiej rozpoznawały klasy pozytywne i neutralne.

```
[165]: print(f"Regresja Logistyczna - Accuracy: {accuracy *100:.2f}%")
print(f"Las drzew - Accuracy: {accuracy_rf *100:.2f}%")
```

Regresja Logistyczna - Accuracy: 74.56%

Las drzew - Accuracy: 74.56%

```
[166]: print("Raport klasyfikacji - Regresja Logistyczna:")
print(classification_report(y_test, predictions, zero_division=0))

print("Raport klasyfikacji - Random Forest:")
print(classification_report(y_test, predictions_rf, zero_division=0))
```

Raport klasyfikacji - Regresja Logistyczna:

	precision	recall	f1-score	support
negatywna	0.75	1.00	0.85	126
neutralna	0.00	0.00	0.00	3
pozytywna	0.00	0.00	0.00	40
accuracy			0.75	169
macro avg	0.25	0.33	0.28	169
weighted avg	0.56	0.75	0.64	169

Raport klasyfikacji - Random Forest:

	precision	recall	f1-score	support
negatywna	0.75	0.99	0.86	126
neutralna	0.00	0.00	0.00	3
pozytywna	0.33	0.03	0.05	40
accuracy			0.75	169
macro avg	0.36	0.34	0.30	169

weighted avg 0.64 0.75 0.65 169

Porównanie modeli Regresji Logistycznej oraz Random Forest wykazało, że model lasu losowego osiągnął nieco wyższą skuteczność klasyfikacji. Model Random Forest uzyskał accuracy na poziomie około 75.15%, natomiast regresja logistyczna około 74.56%.

Oba modele najlepiej rozpoznawały opinie negatywne, natomiast klasy pozytywne i neutralne były klasyfikowane z mniejszą skutecznością. Mogło to wynikać z niewielkiej liczby przykładów tych klas w analizowanym zbiorze danych oraz ograniczeń modelu Word2Vec trenowanego na stosunkowo małej liczbie opinii.

Należy również zauważyć, że model Word2Vec został wytrenowany na stosunkowo niewielkim zbiorze danych, co mogło ograniczyć jakość uzyskanych reprezentacji tekstu oraz wpłynąć na skuteczność klasyfikacji sentymentu.

```
[167]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm_lr = confusion_matrix(y_test, predictions)
cm_rf = confusion_matrix(y_test, predictions_rf)

fig, axes = plt.subplots(1, 2, figsize=(12,5))

# Regresja logistyczna
sns.heatmap(
    cm_lr,
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[0],
    xticklabels=["negatywna", "neutralna", "pozytywna"],
    yticklabels=["negatywna", "neutralna", "pozytywna"]
)

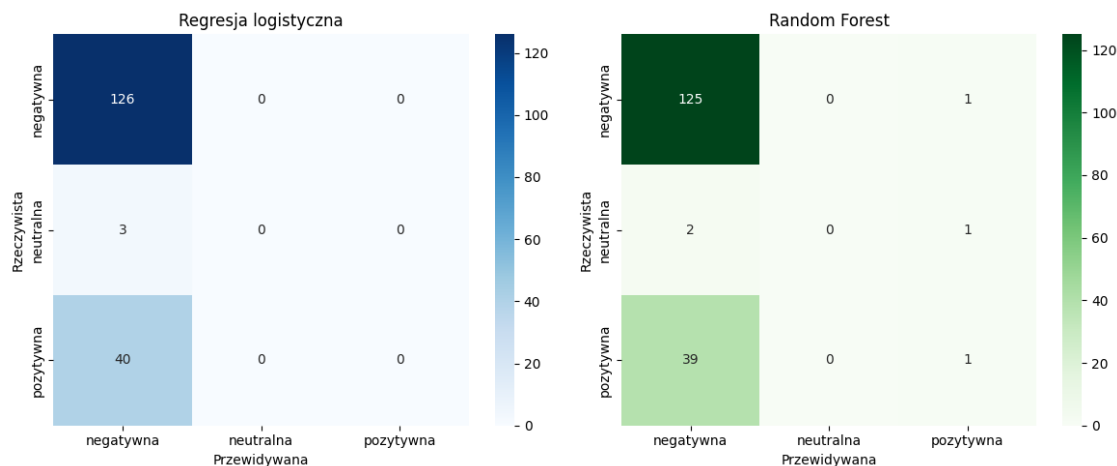
axes[0].set_title("Regresja logistyczna")
axes[0].set_xlabel("Przewidywana")
axes[0].set_ylabel("Rzeczywista")

# Random Forest
sns.heatmap(
    cm_rf,
    annot=True,
    fmt="d",
    cmap="Greens",
    ax=axes[1],
    xticklabels=["negatywna", "neutralna", "pozytywna"],
    yticklabels=["negatywna", "neutralna", "pozytywna"]
)
```

```
)

axes[1].set_title("Random Forest")
axes[1].set_xlabel("Przewidywana")
axes[1].set_ylabel("Rzeczywista")

plt.tight_layout()
plt.show()
```



Wykres 2. Porównanie macierzy pomyłek dla modeli regresji logistycznej oraz Random Forest.

Na podstawie macierzy pomyłek można zauważyć, że oba modele najlepiej rozpoznawały opinie negatywne. Model Random Forest osiągnął nieco lepsze wyniki dla klasy pozytywnej niż regresja logistyczna, jednak oba modele miały trudności z poprawną klasyfikacją opinii neutralnych. Wyniki potwierdzają wpływ niezbalansowania klas na skuteczność modeli klasyfikacyjnych.

3.3.4. Udział opinii według sentymentu

Etykiety sentymentu nie zostały nadane ręcznie, lecz zostały utworzone automatycznie na podstawie oceny końcowej użytkownika. Zadanie klasyfikacyjne polegało na przewidywaniu kategorii sentymentu na podstawie treści opinii tekstowej.

Opinie z oceną do 2 zostały zaklasyfikowane jako negatywne, ocena 3 jako neutralna, natomiast oceny powyżej 3 jako pozytywne.

```
[168]: print(df["Sentyment"].value_counts())
```

```
Sentyment
negatywna    647
pozytywna    179
neutralna     15
Name: count, dtype: int64
```

Z przedstawionych danych wynika, że najwięcej zaklasyfikowanych przez model jest opinii negaty-

wnych w liczbie 647, następnie opinie pozytywne w liczbie 179 i neutralne w liczbie 15.

```
[169]: import matplotlib.pyplot as plt

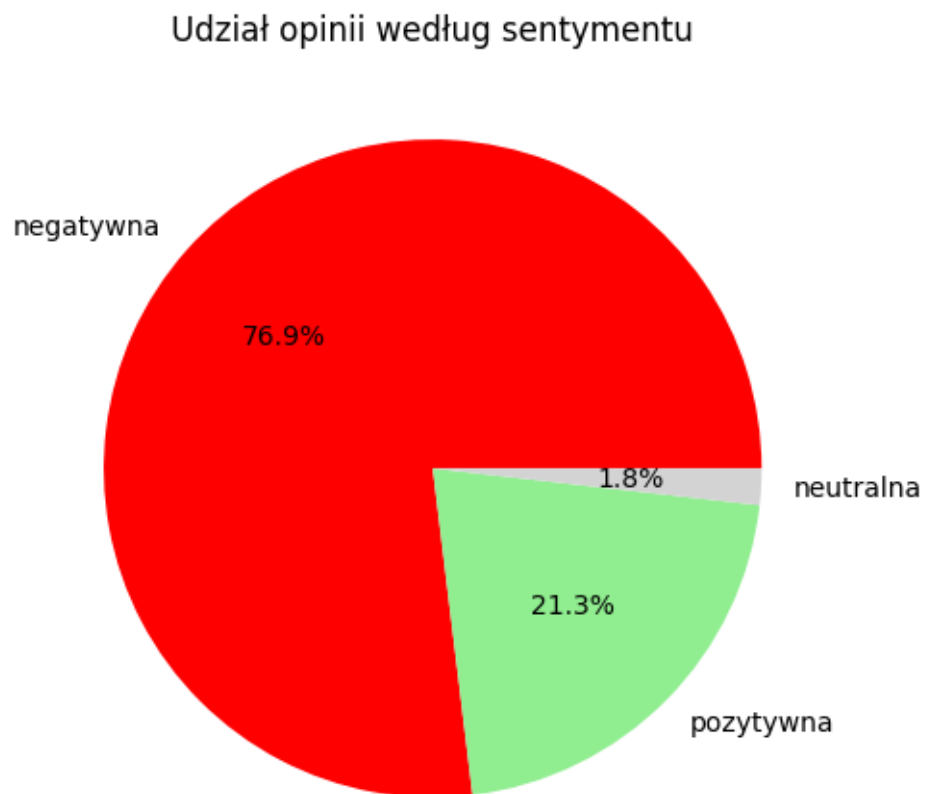
df["Sentyment"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    colors=["red", "lightgreen", "lightgray"]
)

plt.title("Udział opinii według sentymentu")

plt.ylabel("")

plt.tight_layout()

plt.show()
```



Wykres 3. Udział opinii według sentymentu.

Największy udział w zbiorze danych miały opinie negatywne (76.9%). Opinie pozytywne stanowiły 21.3%, natomiast neutralne 1.8%.

3.4. Czynniki wpływające na sentyment opinii

Analiza sentymentu (sentiment analysis), nazywana również analizą wydźwięku, jest jednym z zastosowań przetwarzania języka naturalnego (NLP). Jej celem jest określenie emocjonalnego charakteru tekstu poprzez przypisanie go do odpowiedniej kategorii.

Najczęściej stosowanym podejściem jest klasyfikacja opinii jako: - pozytywnych, - negatywnych, - neutralnych.

Analiza sentymentu znajduje zastosowanie między innymi w analizie opinii klientów, mediach społecznościowych, systemach rekomendacyjnych oraz monitorowaniu opinii o produktach i usługach.

3.4.1. Długość opinii

Długość opinii została wykorzystana jako dodatkowa cecha mogąca wpływać na klasyfikację sentymentu. Założono, że bardziej rozbudowane opinie mogą zawierać większą liczbę informacji dotyczących doświadczeń klientów.

```
[170]: df["Długość Opinii"] = df["Opinia"].str.len()
print(df["Długość_Opinii"].head())
```

```
0      54
1      76
2     296
3     934
4    1194
```

```
Name: Długość_Opinii, dtype: int64
```

Dodano kolumnę określającą długość opinii wyrażoną liczbą znaków. Uzyskana cecha mogła zostać wykorzystana przez model podczas procesu klasyfikacji sentymentu.

3.4.2. Występowanie słowa „problem”

W celu sprawdzenia, czy występowanie określonych słów może wpływać na ocenę klienta, utworzono dodatkową cechę określającą obecność słowa „problem” w opinii.

```
[171]: df["Czy_Problem"] = df["Opinia"].str.contains(
        "problem",
        case=False,
        na=False
    ).astype(int)
print(df["Czy_Problem"].head())
```

```
0      0
1      0
2      0
3      1
```

4 0

Name: Czy_Problem, dtype: int32

Nowa kolumna przyjmowała wartość 1, gdy w opinii występowało słowo „problem”, oraz 0 w pozostałych przypadkach. Cecha ta mogła wskazywać na większe prawdopodobieństwo wystąpienia negatywnego sentymentu.

3.4.3. Analiza sentymentu opinii w zależności od pory dnia

Na podstawie godziny publikacji opinii utworzono cechę określającą porę dnia. Celem było sprawdzenie, czy czas publikacji może mieć związek z charakterem opinii klientów.

```
[172]: Pora_Dnia = []

for godzina in df["Godzina"]:
    if godzina.hour < 12:
        Pora_Dnia.append("Rano")
    elif 12 <= godzina.hour < 18:
        Pora_Dnia.append("Popołudnie")
    else:
        Pora_Dnia.append("Wieczór")

df["Pora_Dnia"] = Pora_Dnia

print(df["Pora_Dnia"].head(10))
```

```
0    Wieczór
1    Wieczór
2    Wieczór
3    Popołudnie
4    Wieczór
5    Wieczór
6    Popołudnie
7     Rano
8    Popołudnie
9     Rano
```

Name: Pora_Dnia, dtype: object

Opinie zostały podzielone na trzy grupy: rano, popołudnie oraz wieczór. Dodana cecha mogła zostać wykorzystana do analizy zależności pomiędzy czasem publikacji opinii a sentymentem.

3.4.4. Analiza najczęstszych problemów klientów

W dalszej części analizy planowane jest określenie najczęściej pojawiających się problemów wskazywanych przez klientów banku ING.

```
[173]: negatywne = df[df["Sentyment"] == "negatywna"]
print(negatywne["Opinia"].head())
```

```
0    proponuję powielanie negatywnych opinii na ins...
1    stosunek do klienta bardzo nieprzyjazny. objaw...
2    obsługa w sosnowcu to dno...czekasz na stojąco...
```

```
3     nie polecam ing.pl, nie mogę zamknąć konta poni...
4     po wstępnej weryfikacji dokumentów ing rozpoczn...
Name: Opinia, dtype: object
```

```
[174]: # Łączenie wszystkich opinii w jeden tekst
tekst = " ".join(negatywne["Opinia"])

# Małe litery
tekst = tekst.lower()

# Usunięcie znaków specjalnych
import re

tekst = re.sub(r"[^a-zA-Ząćłńóśźż\s]", "", tekst)

# Podział na słowa
słowa = tekst.split()

# Usunięcie stop words
słowa = [
    slowo
    for slowo in słowa
    if slowo not in polish_stopwords
]

# Liczenie słów
from collections import Counter

counter = Counter(słowa)

# 10 najczęstszych słów
print(counter.most_common(10))
```

```
[('bank', 459), ('wady', 419), ('zalety', 374), ('banku', 349), ('brak', 341),
('a', 276), ('ing', 238), ('konto', 225), ('konta', 198), ('mi', 156)]
```

W celu identyfikacji najczęściej pojawiających się problemów klientów przeanalizowano najczęściej występujące słowa w opiniach negatywnych. Ze względu na charakter języka naturalnego część często występujących słów nie wносиła istotnej informacji, dlatego zastosowano bigramy.

```
[175]: # Bigramy - analiza najczęstszych problemów klientów

from sklearn.feature_extraction.text import CountVectorizer

# Tylko opinie negatywne
negatywne = df[df["Sentyment"] == "negatywna"]

# CountVectorizer dla bigramów
```



```

vectorizer_bigram = CountVectorizer(
    stop_words=polish_stopwords,
    ngram_range=(2,2)
)

# Zamiana tekstu na macierz bigramów
X_bigram = vectorizer_bigram.fit_transform(negatywne["Opinia"])

# Pobranie nazw bigramów
bigramy = vectorizer_bigram.get_feature_names_out()

# Zliczenie wystąpień
liczby = X_bigram.sum(axis=0)

# Zamiana na zwykłą tablicę
liczby = liczby.A1

# Połączenie: bigram + liczba wystąpień
bigram_freq = list(zip(bigramy, liczby))

# Sortowanie malejąco
bigram_freq = sorted(
    bigram_freq,
    key=lambda x: x[1],
    reverse=True
)

# TOP 10 bigramów
print(bigram_freq[:10])

```

```

[('zalety brak', 196), ('brak wady', 164), ('ten bank', 63), ('tego banku', 60),
('wady brak', 39), ('najgorszy bank', 29), ('wady wszystko', 27), ('szkoda
pisać', 26), ('bank ing', 25), ('wady same', 25)]

```

W pierwszym etapie pojawiły się także bigramy, które nie wносиły istotnych informacji do analizy. Dlatego usunięto wybrane wyrażenia, aby lepiej pokazać najczęstsze problemy klientów.

```

[176]: stop_bigram = [
    "ten bank",
    "tego banku",
    "wady brak",
    "brak wady"
]

bigram_freq = [
    (bigram, liczba)
    for bigram, liczba in bigram_freq
    if bigram not in stop_bigram

```

```
]
print(bigram_freq[:10])
```

```
[('zalety brak', 196), ('najgorszy bank', 29), ('wady wszystko', 27), ('szkoda
pisać', 26), ('bank ing', 25), ('wady same', 25), ('pisać szkoda', 23),
('polecam tego', 22), ('zalety wady', 21), ('banku ing', 18)]
```

Analiza bigramów pozwoliła zidentyfikować najczęściej pojawiające się połączenia słów w opiniach negatywnych klientów. Wśród najczęstszych wyrażeń pojawiały się sformułowania wskazujące na niezadowolenie klientów z jakości usług oraz funkcjonowania banku. Analiza bigramów umożliwiła lepsze zrozumienie najczęściej zgłaszanych problemów klientów.

3.5 Zastosowanie modeli LLM w analizie sentymentu

Duże modele językowe (Large Language Models – LLM) są nowoczesnymi modelami sztucznej inteligencji wykorzystywanymi w przetwarzaniu języka naturalnego. Modele te trenowane są na bardzo dużych zbiorach danych tekstowych, dzięki czemu potrafią rozumieć kontekst wypowiedzi oraz generować tekst przypominający język człowieka.

Większość współczesnych modeli LLM opiera się na architekturze Transformer, która umożliwia analizowanie zależności pomiędzy słowami w całym zdaniu. Do najbardziej znanych modeli należą GPT, BERT oraz LLaMA.

Modele LLM znajdują zastosowanie między innymi w chatbotach, tłumaczeniach automatycznych, analizie sentymentu, generowaniu tekstu oraz wyszukiwaniu informacji. W porównaniu do tradycyjnych metod reprezentacji tekstu, takich jak Bag of Words czy TF-IDF, modele LLM lepiej uwzględniają kontekst i znaczenie wypowiedzi.

```
[177]: #pip install transformers torch sentencepiece
print("Successfully installed: sentencepiece-0.2.1")
```

Successfully installed: sentencepiece-0.2.1

```
[178]: # from transformers import pipeline

# classifier = pipeline(
#     "sentiment-analysis",
#     model="dkleczek/bert-base-polish-uncased-v1"
# )

# classifier("Obsługa banku była bardzo dobra")
print("OSError: [WinError 1114] Procedura inicjowania biblioteki DLL nie_
    <powiodła się.")
```

OSError: [WinError 1114] Procedura inicjowania biblioteki DLL nie powiodła się.

Ze względu na wymagania sprzętowe oraz problemy środowiskowe związane z bibliotekami Transformers i PyTorch, modele LLM zostały przedstawione jedynie w części teoretycznej pracy.

4. Wnioski końcowe

Celem pracy była analiza opinii klientów banku ING z wykorzystaniem metod analizy danych oraz przetwarzania języka naturalnego (NLP). W pracy przeprowadzono proces przygotowania danych, eksploracyjną analizę danych (EDA), analizę sentymentu oraz porównanie wybranych modeli klasyfikacyjnych.

Na podstawie przeprowadzonych analiz można zauważyć, że większość opinii klientów miała charakter negatywny. Najczęściej pojawiające się problemy dotyczyły jakości obsługi klienta, funkcjonowania kont bankowych oraz problemów technicznych. Analiza bigramów pozwoliła lepiej zidentyfikować najczęściej występujące wyrażenia w opiniach klientów.

W części praktycznej zastosowano różne metody reprezentacji tekstu, między innymi Bag of Words, TF-IDF oraz Word2Vec. Najlepsze wyniki uzyskał model wykorzystujący metodę Bag of Words. Porównano również modele Regresji Logistycznej oraz Random Forest. Model Random Forest osiągnął nieco wyższą skuteczność klasyfikacji, jednak oba modele miały trudności z poprawnym rozpoznawaniem klas mniej licznych, szczególnie opinii neutralnych. Głównym ograniczeniem analizy okazało się silne niezbalansowanie klas w zbiorze danych, w którym dominowały opinie negatywne. W związku z tym sama metryka accuracy nie była wystarczająca do pełnej oceny jakości modeli.

W pracy przedstawiono również podstawowe informacje dotyczące dużych modeli językowych (LLM) oraz ich zastosowania w analizie tekstu. W pracy przedstawiono również podstawowe informacje dotyczące dużych modeli językowych (LLM) oraz możliwości ich zastosowania w analizie tekstu i analizie sentymentu.

Przeprowadzona analiza potwierdza, że metody NLP i uczenia maszynowego mogą być skutecznie wykorzystywane do analizy opinii klientów oraz identyfikacji najczęściej występujących problemów. W przyszłości projekt można rozbudować o większy zbiór danych, bardziej zaawansowane metody analizy tekstu oraz dokładniejszą analizę emocji i tematów występujących w opiniach klientów.

Bibliografia

1. Dokumentacja Scikit-learn: <https://scikit-learn.org/>
2. Dokumentacja Pandas: <https://pandas.pydata.org/>
3. Dokumentacja Gensim: <https://radimrehurek.com/gensim/>
4. Dokumentacja Hugging Face Transformers: <https://huggingface.co/docs/transformers/index>
5. Opinie klientów ING Bank Śląski: <https://opiniuj.pl/ing-bank-konto-direct.html>
6. Idczak A., Korzeniewski J., *Statystyczne metody klasyfikacji tekstów*, Uniwersytet Łódzki, Łódź 2022. :contentReferenceoaicite:0
7. Okulska I., „O wiele więcej niż Google Translate, czyli komputerowe przetwarzanie języka naturalnego (NLP) w translatoryce i translatologii”, *Porównania*, 2020. :contentReferenceoaicite:1

Spis wykresów

Wykres 1. Porównanie macierzy pomyłek dla modeli BoW, TF-IDF unigram oraz TF-IDF bigram

Wykres 2. Porównanie macierzy pomyłek dla modeli regresji logistycznej oraz Random Forest

Wykres 3. Udział opinii według sentymentu